## Q1. Creating and cleaning up child processes

1. Create five child processes from a common parent process
2. The parent must create all five children and complete its own work, if any, before calling waitpid() to collect the terminated children.
3. Use waitpid() in the parent to collect each child's termination status.
4. Interpret the exit status of each child and cover normal successful termination, normal unsuccessful termination and abnormal termination.
5. Suggested experiments: make some children run in a loop and allow others to terminate normally. You may use kill -SIGKILL <pid> to terminate a child abnormally.
6. Use ps and top to observe process states during the experiment.

```
desd@desd-OptiPlex-5050:Assignment_3$ gcc Q1.c -o Q1
desd@desd-OptiPlex-5050:Assignment_3$ ./Q1
[Parent] PID: 1455886. Spawning 5 children...
Child 1 PID: 1455887 started.
Child 2 PID: 1455888 started.
Child 3 PID: 1455889 started.
Parent Finished spawning. Doing parent work...
Child 4 PID: 1455890 started.
Child 5 PID: 1455891 started.
Parent Starting to collect terminated children via waitpid()...

Parent Reaped child PID: 1455887
  -> Normal termination. Exit status: 0 Success
Parent Reaped child PID: 1455888
  -> Normal termination. Exit status: 42 Unsuccessful
kill -9 1455889
Parent Reaped child PID: 1455889
  -> Abnormal termination. Killed by signal: 9 (Killed)
Parent Reaped child PID: 1455890
  -> Abnormal termination. Killed by signal: 9 (Killed)
Parent Reaped child PID: 1455891
  -> Abnormal termination. Killed by signal: 9 (Killed)
Parent All children collected. Exiting.
desd@desd-OptiPlex-5050:Assignment_3$ kill -9 1455889
bash: kill: (1455889) - No such process
desd@desd-OptiPlex-5050:Assignment_3$
```

## Q2. Launching applications from child processes

1. Create five child processes from a common parent.
2. Have the children launch different applications, such as a PDF viewer, KWrite, gcc and the application based on while1.c.
3. The parent must call waitpid() after creating all children and completing its own work, if any.
4. Collect and interpret the termination status of every child.
5. Demonstrate successful and unsuccessful normal termination, as well as abnormal termination.

```
desd@desd-OptiPlex-5050:Assignment_3$ gcc Q2.c -o Q2
desd@desd-OptiPlex-5050:Assignment_3$ ./Q2
execlp add failed: No such file or directory
execlp zero failed: No such file or directory
Parent Process (PID: 1457868) successfully spawned 5 calculator child processes.
Parent Process (PID: 1457873) successfully spawned 5 calculator child processes.
Square root: 0
Parent sending SIGKILL to Child 3 (PID: 1457872 - infinite loop calculator) to demonstrate abnormal termination.

--- Collecting and Interpreting Calculator Child Termination Statuses ---

Calculator Child PID 1457869 terminated.
  -> Normal termination.
  -> Exit Status Code: 127
  -> Result: UNSUCCESSFUL termination (non-zero exit code).

Calculator Child PID 1457870 terminated.
  -> Normal termination.
  -> Exit Status Code: 127
  -> Result: UNSUCCESSFUL termination (non-zero exit code).

Calculator Child PID 1457871 terminated.
  -> Normal termination.
  -> Exit Status Code: 0
  -> Result: SUCCESSFUL calculation/process completion.
Parent sending SIGKILL to Child 3 (PID: 1457872 - infinite loop calculator) to demonstrate abnormal termination.

--- Collecting and Interpreting Calculator Child Termination Statuses ---

All calculator processes have been reaped. Parent exiting.

Calculator Child PID 1457872 terminated.
  -> Abnormal termination.
  -> Terminated by signal: 9 (Killed)

Calculator Child PID 1457873 terminated.
  -> Normal termination.
  -> Exit Status Code: 0
  -> Result: SUCCESSFUL calculation/process completion.

All calculator processes have been reaped. Parent exiting.
desd@desd-OptiPlex-5050:Assignment_3$
```

## Q3. Parallel compilation and linking

1. Create five child processes from a common parent.
2. Use the child processes to compile five different source files and generate their respective object files.
3. The parent must use waitpid() to collect the termination status of all children.
4. Based on the exit status information, link the generated object files to create the final executable.
5. If linking succeeds, load and execute the generated binary in a new process.

```
desd@desd-OptiPlex-5050:Assignment_3$ ./40.out
=== Q3.1 & 3.2: Spawning 4 child processes for parallel compilation ===
=== Q3.3: Collecting compilation statuses via waitpid() ===
[SUCCESS] Child PID 1458204 compiled its source file successfully.
[SUCCESS] Child PID 1458205 compiled its source file successfully.
[SUCCESS] Child PID 1458206 compiled its source file successfully.
[SUCCESS] Child PID 1458207 compiled its source file successfully.

=== Q3.4: All compilations passed. Linking object files into 'final_calc'... ===
[SUCCESS] Linking complete! 'final_calc' generated successfully.

=== Q3.5: Loading and Executing 'final_calc' in a new process ===
--- Executing Functions from Linked Modules ---
[add.c] Result of 10 + 20 = 30
[zero.c] Error: Division by zero avoided/handled.
[NonNumeric.c] Square root of 144 is 12
--- Main Program Execution Complete ---
=== Pipeline Completed Successfully. Parent Exiting. ===
desd@desd-OptiPlex-5050:Assignment_3$
```